The Journal of Open Source Software

ffmpeg-webCLI: A Browser-Based Video Editor Powered by WebAssembly

Tejaswi Gowda [1]

1 Arizona State University

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

`ffmpeg-webCLI` is a browser-based video editor that runs the full FFmpeg binary entirely on the client via WebAssembly (Haas et al., 2017). It requires no installation, no server, and no account. All processing happens locally; files never leave the user's device. After the first load, it works completely offline as an installable Progressive Web App (PWA).

The Journal of Open Source Software

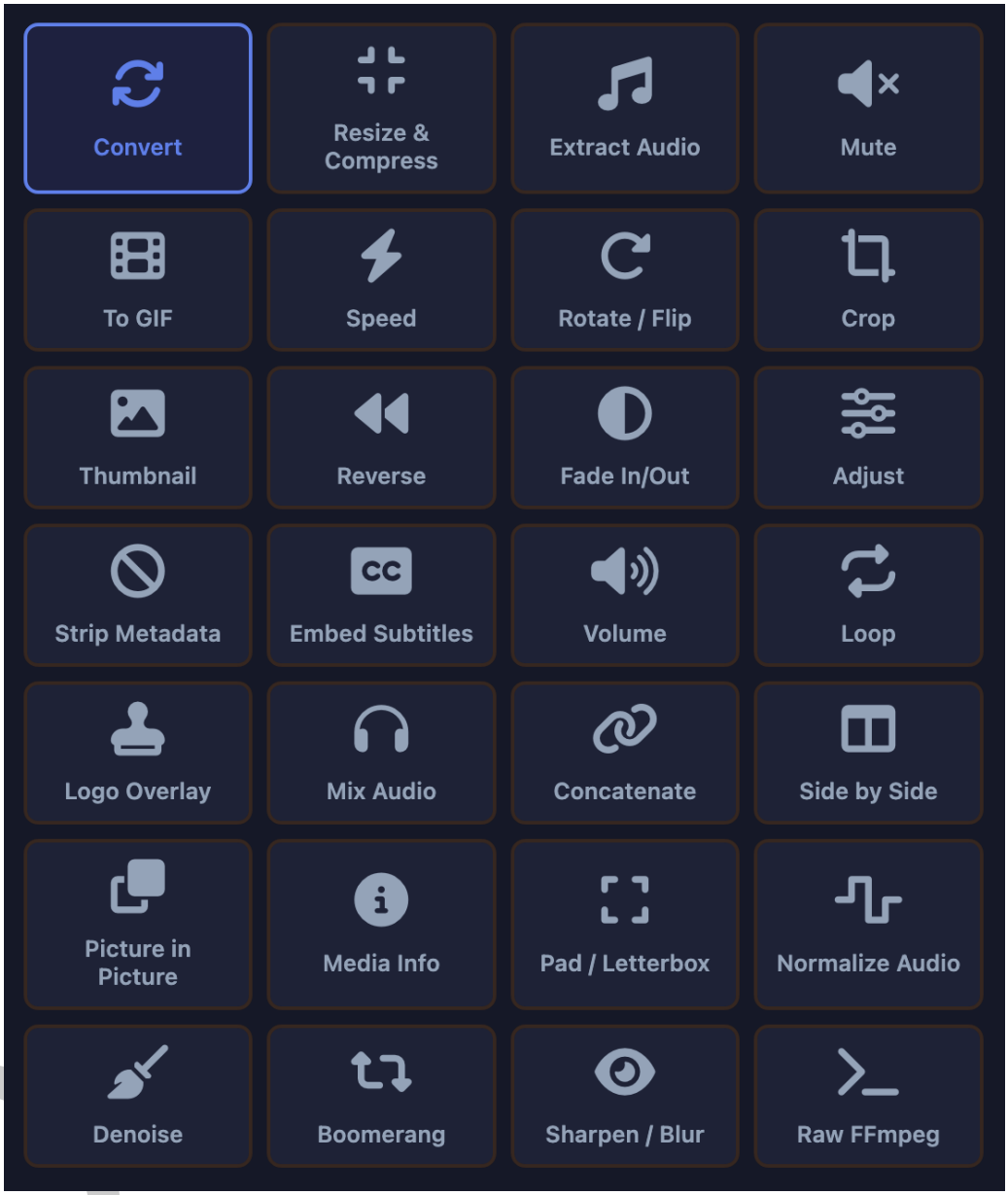

Figure 1: ffmpeg-webCLI interface showing the full operation panel.

The tool exposes 30+ video operations through a graphical interface, including trim, compress, format conversion, GIF creation, audio extraction, subtitle embedding, picture-in-picture, side-by-side composition, logo overlay, audio mixing, denoise, loudness normalization, and pad/letterbox for social formats. It also provides a Raw FFmpeg mode: users type any FFmpeg arguments directly, see a live preview of the exact command that will execute, and run it locally. A collapsible example command library (13 recipes) bridges the two modes for users who know what they want but not the exact syntax.

Statement of Need

FFmpeg is one of the most capable media processing tools ever built. It is also inaccessible to most users: it requires terminal fluency, installation privileges, and knowledge of flag syntax. Existing browser-based alternatives (Kapwing, Clideo, Cloudconvert) solve the interface problem

by uploading files to remote servers. This introduces a privacy tradeoff that is unacceptable for a significant class of use cases: medical video, legal recordings, journalistic source material, and personal footage. For these cases, the privacy guarantee offered by cloud services is a policy document, not a technical property.

`ffmpeg-webCLI` resolves this tradeoff. The WebAssembly sandbox has no network access; data transmission during processing is technically impossible by construction, not just by policy. Any user can verify this by opening browser DevTools and watching zero network requests during processing.

There is also a practical accessibility gap in educational settings. Teaching video production, entertainment engineering, or media arts in a classroom requires software that students can run immediately on any device, without waiting for IT approval or installation. `ffmpeg-webCLI` installs in one click as a PWA and works offline thereafter, removing this barrier entirely.

Architecture

The tool is built on `ffmpeg.wasm` (ffmpegwasm contributors, 2023), which compiles FFmpeg to WebAssembly via Emscripten (Zakai, 2011). Figure 2 shows the overall architecture.

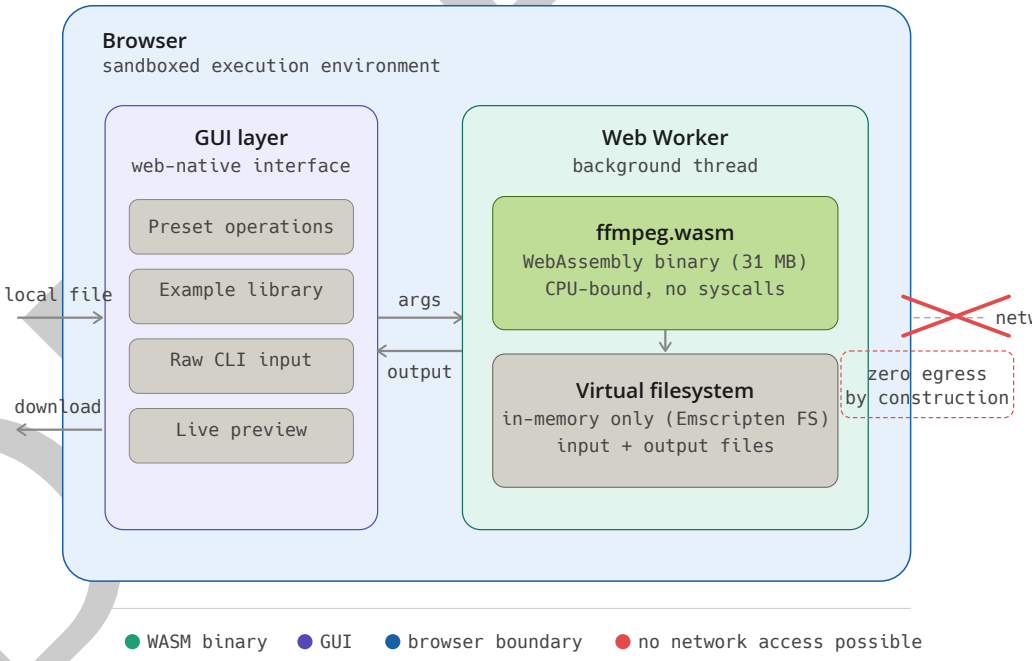

Figure 2: ffmpeg-webCLI architecture: the WASM binary executes in a Web Worker with no network access. Files are read from and written to the user's device only.

The key architectural decisions are:

Web Worker isolation. FFmpeg executes in a dedicated Web Worker, keeping the UI responsive during long encoding operations. Progress is reported via `postMessage` callbacks.

Virtual filesystem. Input files are written to Emscripten's in-memory virtual filesystem (`FS` API) before processing. Output files are read back and offered for download. No data touches the real filesystem or any network endpoint.

Cross-Origin Isolation. `ffmpeg.wasm`'s multi-threaded core requires `SharedArrayBuffer`, which in turn requires Cross-Origin Isolation (`COOP: same-origin` and `COEP: require-corp` headers). A minimal Node.js server (`server.js`) sets these headers for local deployment. The service worker explicitly re-attaches these headers to all cached responses so that Cross-Origin Isolation is preserved in offline mode.

Progressive Web App. A service worker proactively caches the `ffmpeg-core.wasm` binary (31 MB) and all static assets on first load. Subsequent use requires no network access. A Web App Manifest enables installation on desktop and mobile. The Screen Wake Lock API prevents device sleep during long encoding operations.

Progressive disclosure. Three interface levels coexist: (1) preset operations with constrained controls for non-technical users; (2) a collapsible example command library with 13 recipes for users who know what they want; and (3) a raw command interface with live preview for power users who know FFmpeg syntax.

Operations

The following operations are available through the graphical interface:

Category	Operations
Conversion	Format convert (MP4, WebM, MKV, MOV, AVI), GIF maker, Audio extraction (MP3, AAC, WAV, OGG, FLAC)
Editing	Trim, Crop, Rotate/Flip, Resize, Speed change (0.25x–4x), Reverse, Fade in/out
Audio	Mute, Volume (0–4x), Audio replacement, Mix audio, Loop, Normalize loudness (EBU R128 (European Broadcasting Union, 2020))
Enhancement	Brightness/Contrast/Saturation, Grayscale, Strip metadata
Compositing	Logo overlay, Picture-in-picture, Side-by-side, Concatenate
Subtitles	Embed soft subtitle tracks (SRT, VTT, ASS) into MP4 or MKV
Effects	Denoise (hqdn3d, three presets), Sharpen/Blur (unsharp/boxblur), Boomerang
Social	Pad/Letterbox (16:9, 9:16, 1:1, 4:3, 4:5, 21:9)
Advanced	Raw FFmpeg command mode, Example library (13 recipes), Media Info deep scan

Performance

WASM execution is CPU-bound and incurs a mean slowdown of approximately 2–4x relative to native FFmpeg for encoding operations. This is consistent with reported WebAssembly overheads (Jangda et al., 2019), with codec-heavy encoding sitting at the higher end of that range. Stream-copy operations (trim, metadata strip, subtitle embedding) are I/O-bound and show smaller overhead. For clips under five minutes, all operations complete within practical timeframes on modern hardware. This tradeoff is acceptable for the target use case of ad-hoc video processing tasks where privacy and zero installation are more important than throughput.

Community Adoption

Since release, `ffmpeg-webCLI` has received 287 GitHub stars and 31 forks. A public post on Hacker News (Gowda, 2026) received 83 points and 43 comments within 24 hours, with users reporting successful real-world use including legacy format migration (.mpg to .mp4) and classroom deployment as an offline PWA.

Limitations

- Hard subtitle burning requires a `libass`-enabled FFmpeg build, which is not available in the standard `ffmpeg.wasm` binary.
- The WASM memory ceiling (~2 GB) limits processing of very large files.
- No batch processing: files are processed one at a time.
- Encoding is approximately 2–4x slower than native FFmpeg.

Acknowledgements

This project is built on `ffmpeg.wasm` by the ffmpegwasm contributors (ffmpegwasm contributors, 2023), which is itself built on FFmpeg (FFmpeg Developers, 2000). The author thanks the open source community for contributions, feedback, and the first external pull request from @luispa.

References

European Broadcasting Union. (2020). *EBU R 128: Loudness Normalisation and Permitted Maximum Level of Audio Signals*. https://tech.ebu.ch/publications/r128

FFmpeg Developers. (2000). *FFmpeg*. https://ffmpeg.org

ffmpegwasm contributors. (2023). *ffmpeg.wasm: FFmpeg for browser, powered by WebAssembly*. https://github.com/ffmpegwasm/ffmpeg.wasm

Gowda, T. (2026). *Show HN: FFmpeg WebCLI – Full FFmpeg in Browser, Offline PWA, No Uploads (WASM)*. https://news.ycombinator.com/item?id=48404224

Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., & Bastien, J. (2017). Bringing the web up to speed with WebAssembly. *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 185–200. https://doi.org/10.1145/3062341.3062363

Jangda, A., Powers, B., Berger, E. D., & Guha, A. (2019). Not so fast: Analyzing the performance of WebAssembly vs. Native code. *Proceedings of the 2019 USENIX Annual Technical Conference*, 107–120. https://www.usenix.org/conference/atc19/presentation/jangda

Zakai, A. (2011). Emscripten: An LLVM-to-JavaScript compiler. *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion*, 301–312. https://doi.org/10.1145/2048147.2048224